

MODUL 3

KONSEP STRUKTUR DATA, ADT, DAN ANALISIS KOMPLEKSITAS ALGORITMA

1. DESKRIPSI MODUL

Modul praktikum ini membahas konsep dasar struktur data sebagai fondasi utama dalam pengembangan algoritma dan pemrograman yang efisien. Mahasiswa akan diperkenalkan pada cara data disimpan, diorganisasikan, dan dikelola agar proses pengolahan data dapat dilakukan secara optimal.

Selain itu, mahasiswa juga akan mempelajari konsep **Abstract Data Type (ADT)** sebagai representasi konseptual dari suatu struktur data, serta pentingnya analisis efisiensi algoritma menggunakan pendekatan **Big-O Notation**.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami hubungan antara struktur data, algoritma, dan efisiensi program dalam menyelesaikan permasalahan komputasi.

2. TUJUAN PRAKTIKUM

Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep dasar struktur data dan efisiensi algoritma.

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Memahami konsep dasar struktur data
- Menjelaskan pengertian ADT (Abstract Data Type)
- Memahami pentingnya efisiensi algoritma
- Mengenali kompleksitas waktu algoritma
- Menggunakan Big-O Notation sederhana
- Membandingkan efisiensi beberapa algoritma dasar

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep struktur data dan ADT
- Mengidentifikasi efisiensi suatu algoritma
- Menganalisis kompleksitas waktu sederhana
- Membandingkan performa algoritma
- Mengimplementasikan algoritma sederhana menggunakan Python

4. DASAR TEORI

4.1 Struktur Data

Struktur data adalah cara untuk menyimpan, mengorganisasikan, dan mengelola data di dalam komputer sehingga data dapat digunakan secara efisien.

Pemilihan struktur data yang tepat sangat penting karena akan mempengaruhi:

- Kecepatan pemrosesan data
- Efisiensi penggunaan memori
- Kemudahan pengembangan program

Contoh struktur data:

- Array
- List
- Stack
- Queue
- Tree
- Graph

Contoh Sederhana:

```
data = [10, 20, 30, 40]  
print(data)
```

Pada contoh di atas, data disimpan dalam bentuk list sehingga dapat diakses secara terstruktur.

4.2 Jenis Struktur Data

Secara umum struktur data dibagi menjadi dua kelompok:

1. Struktur Data Linear

Data disusun secara berurutan.

Contoh:

- Array
- List
- Stack
- Queue

2. Struktur Data Non-Linear

Data tidak disusun secara berurutan melainkan berbentuk hierarki atau jaringan.

Contoh:

- Tree
- Graph

4.3 Abstract Data Type (ADT)

Abstract Data Type (ADT) adalah model konseptual yang mendefinisikan:

- Jenis data
- Operasi yang dapat dilakukan
- Perilaku struktur data

ADT tidak fokus pada implementasi, melainkan pada konsep dan fungsi.

Contoh ADT Stack

Operasi utama:

- Push → menambah data
- Pop → mengambil data
- Peek → melihat data teratas

Contoh Implementasi:

```
stack = []  
  
stack.append(10)  
stack.append(20)  
  
print(stack.pop())
```

4.4 Pentingnya Efisiensi Algoritma

Efisiensi algoritma menunjukkan seberapa cepat dan seberapa hemat memori suatu program dijalankan.

Algoritma yang baik:

- Cepat diproses
- Menggunakan memori secara optimal
- Mampu menangani data besar

Contoh:

Mencari data dalam:

- List kecil → cepat
- List sangat besar → membutuhkan algoritma efisien

4.5 Kompleksitas Algoritma

Kompleksitas algoritma digunakan untuk mengukur efisiensi suatu algoritma.

Terdapat dua jenis kompleksitas:

1. Kompleksitas waktu (*time complexity*)
2. Kompleksitas ruang (*space complexity*)

4.6 Big-O Notation

Big-O digunakan untuk menggambarkan pertumbuhan waktu eksekusi algoritma terhadap jumlah data.

Jenis Kompleksitas Umum

Kompleksitas	Nama	Contoh
$O(1)$	Konstan	Akses elemen array
$O(\log n)$	Logaritmik	Binary Search
$O(n)$	Linear	Pencarian linear
$O(n^2)$	Kuadratik	Bubble Sort

Contoh $O(n)$

```
data = [1, 2, 3, 4, 5]
```

```
for x in data:  
    print(x)
```

Perulangan berjalan sebanyak jumlah data.

Contoh $O(n^2)$

```
for i in range(5):  
    for j in range(5):  
        print(i, j)
```

Karena terdapat perulangan di dalam perulangan, kompleksitas menjadi $O(n^2)$.

4.7 Perbandingan Efisiensi Algoritma

Efisiensi algoritma sangat penting terutama ketika jumlah data besar.

Contoh:

- Linear Search → mencari satu per satu
- Binary Search → membagi data menjadi dua bagian

Binary Search lebih cepat dibanding Linear Search untuk data yang besar dan terurut.

Contoh Linear Search

```
data = [10, 20, 30, 40]
target = 30
```

```
for i in data:
    if i == target:
        print("Data ditemukan")
```

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python (Google Colab / IDE)
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

6.1 Struktur Data Dasar

Langkah Praktikum:

1. Buka aplikasi Python
2. Buat list sederhana
3. Tampilkan isi data
4. Akses elemen tertentu

Contoh Program:

```
data = [10, 20, 30, 40]

print(data)
print(data[0])
```

Tujuan:

Mahasiswa memahami konsep penyimpanan data terstruktur.

6.2 Abstract Data Type (ADT)

Langkah Praktikum:

1. Buat stack sederhana menggunakan list
2. Tambahkan data menggunakan push
3. Ambil data menggunakan pop

Contoh Program:

```
stack = []

stack.append(10)
stack.append(20)

print(stack.pop())
```

Tujuan:

Mahasiswa memahami konsep ADT Stack.

6.3 Analisis Kompleksitas Linear

Langkah Praktikum:

1. Buat list data
2. Gunakan perulangan untuk traversal
3. Hitung jumlah iterasi

Contoh Program:

```
data = [1, 2, 3, 4, 5]

for i in data:
    print(i)
```

Tujuan:

Mahasiswa memahami kompleksitas $O(n)$.

6.4 Analisis Kompleksitas Kuadratik

Langkah Praktikum:

1. Gunakan nested loop
2. Jalankan program
3. Amati jumlah proses yang meningkat

Contoh Program:

```
for i in range(3):  
    for j in range(3):  
        print(i, j)
```

Tujuan:

Mahasiswa memahami kompleksitas $O(n^2)$.

6.5 Perbandingan Algoritma**Langkah Praktikum:**

1. Buat algoritma pencarian sederhana
2. Bandingkan jumlah proses
3. Diskusikan efisiensinya

Contoh Program:

```
data = [10, 20, 30, 40]  
target = 30  
  
for i in data:  
    if i == target:  
        print("Ditemukan")
```

Tujuan:

Mahasiswa memahami pentingnya algoritma efisien.

7. STUDI KASUS**Studi Kasus 1: Penyimpanan Data Mahasiswa****Deskripsi:**

Sebuah sistem akademik ingin menyimpan daftar nama mahasiswa.

Permasalahan:

Buat program untuk:

- Menyimpan data mahasiswa dalam list
- Menampilkan seluruh data

Tujuan:

Memahami penggunaan struktur data sederhana.

Studi Kasus 2: Simulasi Stack Buku

Deskripsi:

Tumpukan buku di perpustakaan mengikuti konsep LIFO.

Permasalahan:

Buat simulasi:

- Menambahkan buku
- Mengambil buku paling atas

Tujuan:

Memahami konsep ADT Stack.

Studi Kasus 3: Analisis Perulangan

Deskripsi:

Dosen ingin menunjukkan perbedaan jumlah proses pada loop biasa dan nested loop.

Permasalahan:

Bandingkan:

- Single loop
- Nested loop

Tujuan:

Memahami kompleksitas algoritma.

Studi Kasus 4: Pencarian Data

Deskripsi:

Sebuah koperasi ingin mencari ID anggota tertentu.

Permasalahan:

Buat program pencarian sederhana menggunakan linear search.

Tujuan:

Memahami efisiensi algoritma pencarian.

8. TUGAS PRAKTIKUM

LEVEL MUDAH

Tugas 1

Buat program untuk:

- Menyimpan 5 angka dalam list
- Menampilkan seluruh data

Tugas 2

Buat simulasi stack sederhana menggunakan list.

Tugas 3

Buat program traversal data menggunakan perulangan.

LEVEL SEDANG

Tugas 4

Buat program untuk menghitung jumlah iterasi dalam loop.

Tugas 5

Buat program nested loop dan analisis kompleksitasnya.

Tugas 6

Buat simulasi pencarian data sederhana.

LEVEL SULIT

Tugas 7

Bandingkan efisiensi dua algoritma pencarian sederhana.

Tugas 8

Buat program simulasi sistem antrian mahasiswa.

Tugas 9

Analisis kompleksitas beberapa algoritma yang dibuat.

9. DISKUSI

- a. Mengapa struktur data penting dalam pemrograman?
- b. Apa perbedaan struktur data linear dan non-linear?
- c. Mengapa algoritma harus efisien?
- d. Apa fungsi Big-O Notation?
- e. Mengapa nested loop lebih lambat dibanding single loop?

10. FORMAT LAPORAN PRAKTIKUM

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil (Screenshot Program)
6. Analisis
7. Kesimpulan

11. RUBRIK PENILAIAN

Komponen	Bobot
Kehadiran	10%
Kode Program	30%
Hasil	30%
Analisis	30%